

APIs as infrastructure: future-proofing Stripe with versioning

August 5, 2017



Brandur Leach
API Experience

When it comes to APIs, change isn't popular. While software developers are used to iterating quickly and often, API developers lose that flexibility as soon as even one user starts consuming their interface. Many of us are familiar with how the Unix operating system evolved. In 1994, *The Unix-Haters Handbook* was published containing a long list of missives about the software—everything from overly-cryptic command names that were optimized for Teletype machines, to irreversible file deletion, to unintuitive programs with far too many options. Over twenty years later, an overwhelming majority of these complaints are still valid even across the dozens of modern derivatives. Unix had become so widely used that changing its behavior would have challenging implications. For better or worse, it established a contract with its users that defined how Unix interfaces behave.

Similarly, an API represents a contract for communication that can't be changed without considerable cooperation and effort. Because so many businesses rely on Stripe as infrastructure, we've been thinking about these contracts since Stripe started. To date, we've maintained compatibility with every version of our API since the company's inception in 2011. In this article, we'd like to share how we manage API versions at Stripe.

Code written to integrate with an API has certain inherent expectations built into it. If an endpoint returns a boolean field called `verified` to indicate the status of a bank account, a user might write code like this:



If we later replaced the bank account's `verified` boolean with a `status` field that might include the value `verified` (like we did back in 2014), the code will break because it depends on a field that no longer exists. This type of change is *backwards-incompatible*, and we avoid making them. Fields that were present before should stay present, and fields should always preserve their same type and name. Not all changes are backwards-incompatible though; for example, it's safe to add a new API endpoint, or a new field to an existing API endpoint that was never present before.

With enough coordination, we might be able to keep users apprised of changes that we're about to make and have them update their integrations, but even if that were possible, it wouldn't be very user-friendly. Like a connected power grid or water supply, after hooking it up, an API should run without interruption for as long as possible.

Our mission at Stripe is to provide the economic infrastructure for the internet. Just like a power company shouldn't change its voltage every two years, we believe that our users should be able to trust that a web API will be as stable as possible.

API versioning schemes

A common approach to allow forward progress in web APIs is to use versioning. Users specify a version when they make requests and API providers can make the changes they want for their next version while maintaining compatibility in the current one. As new versions are released, users can upgrade when it's convenient for them.

This is often seen as a major versioning scheme with names like `v1`, `v2`, and `v3` that are passed as a prefix to a URL (like `/v1/widgets`) or through an HTTP header like `Accept`. This can work, but has the major downside of changes between versions being so big and so impactful for users that it's almost as painful as re-integrating from scratch. It's also not a clear win because there will be a class of users that are unwilling or unable to upgrade and get trapped on old API versions. Providers then have to make the difficult choice between retiring API versions and by extension cutting those users off, or maintaining the old versions forever at considerable cost. While having providers maintain old versions might seem at first glance to be beneficial to users, they're also paying indirectly in the form of reduced progress on improvements. Instead of working on new features, engineering time is diverted to maintaining old code.

At Stripe, we implement versioning with rolling versions that are named with the date they're released (for example, `2017-05-24`). Although backwards-incompatible, each one contains a small set of changes that make incremental upgrades relatively easy so that integrations can stay current.

The first time a user makes an API request, their account is automatically pinned to the most recent version available, and from then on, every API call they make is assigned that version implicitly. This approach guarantees that users don't accidentally receive a breaking change and makes initial integration less painful by reducing the amount of necessary configuration. Users can override the version of any single request by manually setting the `Stripe-Version` header, or upgrade their account's pinned version from Stripe's dashboard.

Some readers might have already noticed that the Stripe API also defines major versions using a prefixed path (like `/v1/charges`). Although we reserve the right to make use of this at some point, it's not likely to change for some time. As noted above, major version changes tend to make upgrades painful, and it's hard for us to imagine an API redesign that's important enough to justify this level of user impact. Our current approach has been sufficient for almost a hundred backwards-incompatible upgrades over the past six years.

Versioning under the hood

Versioning is always a compromise between improving developer experience and the additional burden of maintaining old versions. We strive to achieve the former while minimizing the cost of the latter, and have implemented a versioning system to help us with it. Let's take a quick look at how it works. Every possible response from the Stripe API is codified by a class that we call an *API resource*. API resources define their possible fields using a DSL:

~ 

API resources are written so that the structure they describe is what we'd expect back from the current version of the API. When we need to make a backwards-incompatible change, we encapsulate it in a *version change module* which defines documentation about the change, a transformation, and the set of API resource types that are eligible to be modified:

~ 

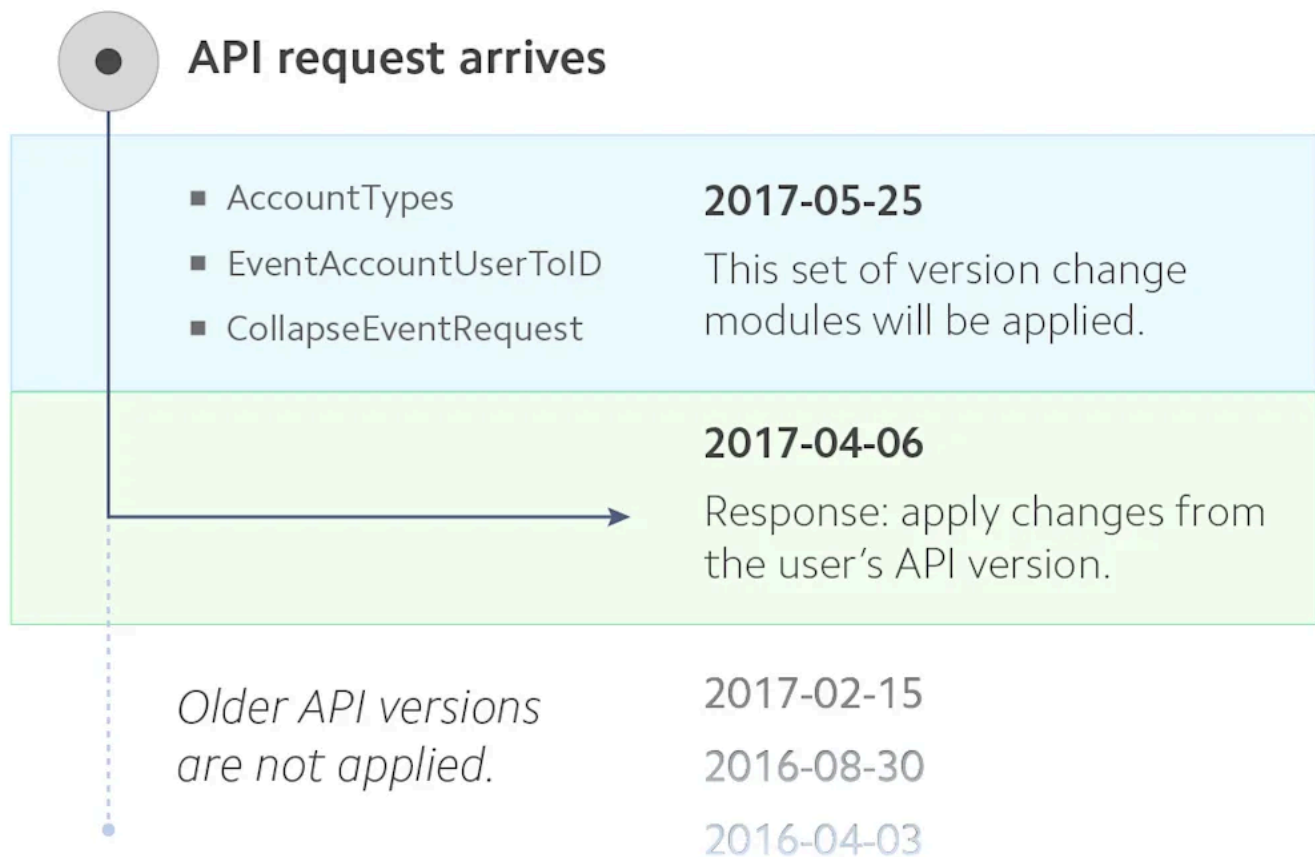
Elsewhere, version changes are assigned to a corresponding API version in a master list:

Version changes are written so that they expect to be automatically applied backwards from the current API version and in order. Each version change assumes that although newer changes may exist in front of them, the data they receive will look the same as when they were originally written.

When generating a response, the API initially formats data by describing an API resource at the current version, then determines a target API version from one of:

- A `Stripe-Version` header if one was supplied.
- The version of an authorized OAuth application if the request is made on the user's behalf.
- The user's pinned version, which is set on their very first request to Stripe.

It then walks back through time and applies each version change module that finds along the way until that target version is reached.



Requests are processed by version change modules before returning a response.

Version change modules keep older API versions abstracted out of core code paths. Developers can largely avoid thinking about them while they're building new products.

Changes with side effects

Most of our backwards-incompatible API changes will modify a response, but that's not always the case. Sometimes a more complicated change is necessary which leaks out of the module that defines it. We assign these modules a `has_side_effects` annotation and the transformation they define becomes a no-op:



Elsewhere in the code a check will be made to see whether they're active:



This reduced encapsulation makes changes with side effects more complex to maintain, so we try to avoid them.

Declarative changes

One advantage of self-contained version change modules is that they can declare documentation describing what fields and resources they affect. We can also reuse this to rapidly provide more helpful information to our users. For example, [our API changelog](#) is programmatically generated and receives updates as soon as our services are deployed with a new version.

We also tailor our API reference documentation to specific users. It notices who is logged in and annotates fields based on their account API version. Here, we're warning the developer that there's been a backwards-incompatible change in the API since their pinned version. The `request` field of Event was previously a string, but is now a subobject that also contains an idempotency key (produced by the version change that we showed above):

⚠ request Information on the API request that

Changes since your API version (2017-04-06):

Event objects (and webhooks) will now render a **request** subobject that contains a request ID and idempotency key instead of just a string request ID.

of the event: e.g. **request.id**, **request.idempotency_key**, **request.metadata**, **request.refunded**, etc.

Our documentation detects the user's API version and presents relevant warnings.

Minimizing change

Providing extensive backwards compatibility isn't free; every new version is more code to understand and maintain. We try to keep what we write as clean as possible, but given enough time dozens of checks on version changes that can't be encapsulated cleanly will be littered throughout the project, making it slower, less readable, and more brittle. We take a few measures to try and avoid incurring this sort of expensive technical debt.

Even with our versioning system available, we do as much as we can to avoid using it by trying to get the design of our APIs right the first time. Outgoing changes are funneled through a lightweight API review process where they're written up in a brief supporting document and submitted to a mailing list. This gives each proposed change broader visibility throughout the company, and improves the likelihood that we'll catch errors and inconsistencies before they're released.

We try to be mindful of balancing stagnation and leverage. Maintaining compatibility is important, but even so, we expect to eventually start retiring our older API versions. Helping users move to newer versions of the API gives them access to new features, and simplifies the foundation that we use to build new features.

Principles of change

The combination of rolling versions and an internal framework to support them has enabled us to onboard vast numbers of users, make enormous changes to our API—all while having minimal impact on existing integrations. The approach is driven by a few principles that we've picked up over the years. We think it's important that API upgrades are:

- **Lightweight.** Make upgrades as cheap as possible (for users and for ourselves).
- **First-class.** Make versioning a first-class concept in your API so that it can be used to keep documentation and tooling accurate and up-to-date, and to generate a changelog automatically.
- **Fixed-cost.** Ensure that old versions add only minimal maintenance cost by tightly encapsulating them in version change modules. Put another way, the less thought that needs to be applied towards old behavior while writing new code, the better.

While we're excited by the debate and developments around REST vs. GraphQL vs. gRPC, and—more broadly—what the future of web APIs will look like, we expect to continue supporting versioning schemes for a long time to come.

Subscribe to the Stripe Blog

Stay connected with Stripe and receive new blog posts in your inbox.

Subscribe >

Stripe will handle your data pursuant to its [Privacy Policy](#)



Like this post? Join our team.

Stripe builds financial tools and economic infrastructure for the internet.



Have any feedback or questions?

We'd love to hear from you.

View roles >

stripe

📍 United States (English)

Products & pricing

Pricing

Atlas

Authorization Boost

Billing

Capital

Checkout

Crypto

Climate

Connect

Data Pipeline

Elements

Treasury

Financial Connections

Identity

Invoicing

Issuing

Link

Managed Payments

Payments

Payment links

Payouts

Radar

Revenue Recognition

Stripe Sigma

Tax

Terminal

Solutions

Enterprises

Startups

Agentic commerce

Crypto

Ecommerce

Embedded finance

Finance automation

Global businesses

In-app payments

Marketplaces

Platforms

SaaS

AI companies

Creator economy

Hospitality, travel, and leisure

Insurance

Media and entertainment

Nonprofits

Public sector

Retail

Integrations & custom solutions

Stripe App Marketplace

Stripe Partner ecosystem

Professional services

Developers

Documentation

API reference

API status

API changelog

Libraries and SDKs

Developer blog

Resources

Guides

Customer stories

Blog

Sessions annual conference

Privacy & terms

Prohibited & restricted businesses

Licenses

Sitemap

Cookie settings

Your privacy choices

More resources

Company

Product roadmap

Jobs

Newsroom

Stripe Press

Contact sales

Support

Get support

Managed support plans

CA residents: +1 888 926 2289

Sign in >

© 2026 Stripe, LLC

